AMENDIS / VEOLIA, SERVICE RH

Portail RH Documents

Guide de prise en main pour nouvel ingénieur

Contexte, fonctionnalités, état d'avancement et démarrage rapide

Objet de ce document

Ce document vous donne une vue d'ensemble du projet avant de plonger dans le code : ce qu'il fait, ce qui est déjà construit, la stack technique, comment lancer le projet en local, et où trouver le reste de la documentation. Pour le détail technique complet (architecture, API, modèle de données, conventions), la référence vivante est `doc/TECHNICAL.md`. Lisez-le en premier une fois cette vue d'ensemble digérée.

Table des matières

1 Vue d'ensemble 1

2 Fonctionnalités principales 2

3 Stack technique 2

4 État d'avancement 2

5 Démarrage rapide 3

6 Configurer la synchronisation Drive (Google Apps Script) 3

7 Les branches 4

8 Documentation 5

9 Pour bien démarrer 5

1 Vue d'ensemble

Le **Portail RH Documents** est une application web interne qui donne aux équipes RH un accès contrôlé aux documents des salariés stockés sur **Google Drive**, sans que les agents aient à fouiller manuellement dans l'arborescence Drive, et sans leur donner un accès plus large que ce dont ils ont besoin.

Principe d'architecture central :

Couche	Rôle
Google Drive	Stockage des fichiers. Reste la source de vérité, les PDF n'y sont jamais copiés ailleurs.
PostgreSQL	Index des métadonnées, des utilisateurs et des permissions. C'est sur cette base que la recherche s'exécute, jamais sur Drive en direct.
Application Next.js	Authentification, contrôle d'accès, recherche, interface RH/admin.

Un document refusé à un utilisateur apparaît dans l'interface **grisé, sans lien**. Il n'est jamais masqué en front uniquement : l'accès est toujours vérifié côté serveur.

Contact : **Abdellah Raissouni**

2 Fonctionnalités principales

— **Authentification et comptes** : login, mot de passe oublié, gestion des utilisateurs et des rôles (`ADMIN`, `RH_MANAGER`, `RH_AGENT`)

— **Recherche documentaire avancée** : recherche plein texte sur toutes les colonnes de la feuille Base (CIN, diplôme, zone, responsable, etc.), filtres par identité, organisation, dates, avec export **Excel** et export **ZIP** (arborescence par salarié/catégorie) des résultats

— **Annuaire et dossier salarié** : fiche par matricule avec état civil, documents indexés et liens Drive (si autorisé)

— **Complétude des dossiers** : checklist par salarié des types de documents attendus vs indexés, avec un mécanisme d'attestation manuelle pour les cas non résolubles automatiquement

— **Reporting** : tableau de bord démographie de l'effectif et vue d'ensemble de la complétude

— **Permissions granulaires** : matrice par rôle/utilisateur × catégorie de dossier, plus une portée par salarié (direction/service)

— **Synchronisation Drive** : un Google Apps Script pousse l'arborescence Drive vers le portail. **Le portail n'appelle jamais l'API Drive lui-même** (voir §6)

— **Sécurité** : mots de passe chiffrés, limitation de débit, en-têtes de sécurité, webhooks signés HMAC

3 Stack technique

Couche	Choix
Framework	Next.js 16 (App Router, Turbopack)
Langage	TypeScript
UI	Tailwind CSS v4, Shadcn/ui, Framer Motion
Formulaires	react-hook-form + zod
Auth	NextAuth.js v5 (Credentials, JWT)
Base de données	PostgreSQL 16+, Prisma 7
Email	Nodemailer (reset password ; Ethereal en dev)
Synchronisation Drive	Google Apps Script (push vers le portail, aucune API Drive côté serveur)
Graphiques	recharts (page reporting)

4 État d'avancement

Ce qui est déjà construit

- ☐ Authentification, gestion des comptes, paramètres utilisateur
- ☐ Synchronisation Drive réelle via Google Apps Script (webhook signé, testée de bout en bout)
- ☐ Recherche documentaire avancée avec exports Excel et ZIP
- ☐ Annuaire, dossier salarié, complétude des dossiers avec attestations manuelles
- ☐ Reporting : démographie de l'effectif et vue de complétude
- ☐ Permissions : matrice par rôle/utilisateur et portée par salarié (direction/service)
- ☐ Sécurité : chiffrement, limitation de débit, en-têtes, webhooks HMAC

5 Démarrage rapide

1. **Prérequis** : Node.js **20.9+**, PostgreSQL **16+** accessible localement, npm.
2. Cloner le dépôt et installer les dépendances :

```
git clone <url-du-depot>
cd projet-rh-annexe
npm install
```

3. Copier `.env.example` vers `.env` et renseigner au minimum `DATABASE_URL` et `AUTH_SECRET` (`openssl rand -base64 32`). Les variables SMTP et synchronisation Drive peuvent rester vides en local.
4. Appliquer le schéma Prisma à la base et créer le compte admin :

```
npm run db:setup
```

Sortie réelle observée (schéma déjà à jour, seed idempotent, peut être relancé sans risque) :

```
> projet-rh-annexe@0.1.0 db:setup
> prisma db push && tsx prisma/seed.ts

Loaded Prisma config from prisma.config.ts.
Prisma schema loaded from prisma\schema.prisma.
Datasource "db": PostgreSQL database "rh_annexe", schema "public" at "localhost:5433"

The database is already in sync with the Prisma schema.

Seed OK -- admin@amendis.ma / Admin123!
```

5. Lancer le serveur de développement :

```
npm run dev
```

6. Ouvrir `http://localhost:3000`. Connexion : `admin@amendis.ma` / `Admin123!`

> **Piège fréquent en local**
>
> Sur certaines machines Windows avec PostgreSQL 18, l'instance locale tourne sur le port **5433** et non le 5432 par défaut. À vérifier avant de chercher une erreur de connexion ailleurs (remarquez le port 5433 dans la sortie ci-dessus).

Pour ce qu'il faut fournir côté IT pour une mise en production (hébergement, base de données, SMTP, variables d'environnement), voir `doc/DEPLOYMENT.md`.

6 Configurer la synchronisation Drive (Google Apps Script)

Cette étape n'est **pas nécessaire pour développer en local** (le point précédent suffit pour lancer le portail avec une base vide ou déjà peuplée). Elle concerne la connexion à un vrai Google Drive RH. Référence complète : `doc/APPS_SCRIPT_SYNC_SETUP.md` ; script source : `scripts/apps-script/sync.gs` dans le dépôt.

> **Aucune API Drive, aucun compte de service**
>
> Le portail ne détient **aucune** clé Google, ni compte de service, ni client OAuth. Un Google Apps Script (un script qui s'exécute *dans* Google, sous l'identité Workspace de l'entreprise) lit le Drive et pousse les données vers le portail. Le portail ne fait jamais l'inverse, sauf pour l'export ZIP (voir `doc/TECHNICAL.md` §9).

1. **Localiser le script.** Le code à déployer est dans le dépôt : `scripts/apps-script/sync.gs`. Ce fichier n'est **pas** exécuté par Next.js : c'est un fichier source à copier manuellement dans un projet Google Apps Script.
2. **Créer le projet Apps Script.** Depuis un compte Google Workspace de l'entreprise, aller sur `script.google.com` et créer un nouveau projet. Plus simple : ouvrir la feuille « Base » (feuille Excel/Sheets

contenant les salariés) et utiliser `Extensions>AppsScript`, ce qui lie automatiquement le script à cette feuille.

3. **Coller le code.** Copier tout le contenu de `scripts/apps-script/sync.gs` dans l'éditeur Apps Script (remplacer le fichier `Code.gs` par défaut).
4. **Renseigner les propriétés du script.** Dans l'éditeur : `ParamÃtresduprojet(icÃt'neengrenage)>PropriÃl'tÃl'sduscript>AjouterunepropriÃl'tÃl'duscript`. Ajouter les quatre valeurs suivantes :

```
PORTAL_WEBHOOK_URL  = https://<votre-domaine>/api/sync/apps-script
SYNC_SECRET         = <valeur longue aleatoire, ex. openssl rand -base64 32>
ROOT_FOLDER_ID      = <ID du dossier Drive racine, un sous-dossier par matricule>
BASE_SHEET_ID       = <ID de la feuille "Base" (salaries)>
```

   L'ID d'un dossier ou d'une feuille Google est la partie de l'URL après `/folders/` ou `/d/`.
5. **Synchroniser le secret côté portail.** La variable d'environnement `DRIVE_SYNC_WEBHOOK_SECRET` du portail doit être **exactement égale** à `SYNC_SECRET` ci-dessus. Utiliser une valeur différente par environnement (dev, staging, prod) : ne jamais réutiliser le même secret.
6. **Principe du moindre privilège.** Exécuter le script sous un compte Workspace qui n'a accès qu'au dossier racine RH et à la feuille Base, pas un accès Drive global à tout le domaine.
7. **Tester manuellement.** La fonction `testRunSyncManually` est fournie dans le script pour lancer une synchronisation à la demande depuis l'éditeur (bouton « Exécuter »), avant de programmer le déclencheur automatique.
8. **Programmer le déclencheur.** Dans l'éditeur Apps Script : `DÃl'clencheurs(icÃt'nehorloge)>AjouterundÃl'clencheur`, fonction `runSync`, type d'événement `BasÃl'surletemps`, fréquence `Toutesles30minutes` (ajustable). C'est ce déclencheur, pas un bouton dans le portail, qui lance les synchronisations en production. L'interface `/admin/sync` du portail est en lecture seule.

> **Pièges déjà rencontrés en test réel**
>
> Voir `doc/APPS_SCRIPT_SYNC_SETUP.md` pour le détail, notamment : un nom d'onglet `"Base"` avec un espace final fait échouer `getSheetByName("Base")` silencieusement ; la ligne 1 de la feuille Base doit être vide, les en-têtes en ligne 2 ; et la signature HMAC doit être calculée en UTF-8 explicite pour ne pas échouer sur les caractères accentués (`PrÃl'nom`, `Dated'EntrÃl'e`, etc.).

7 Les branches

Le développement se fait sur des branches de fonctionnalité fusionnées dans `main` une fois stabilisées.

> **main, la branche du projet**
>
> C'est **la branche de référence**, celle sur laquelle s'appuyer pour démarrer. Elle contient l'architecture actuelle et validée : synchronisation Drive via **Google Apps Script** (le portail ne parle jamais directement à l'API Google Drive, voir §1 et §6), recherche PostgreSQL, permissions, exports, et toute la documentation à jour dans `doc/`. Sauf indication contraire, c'est de là qu'il faut partir.

Une branche annexe, `feature/local-server-migration`, explore une direction différente : abandonner Google Drive au profit d'un stockage **auto-hébergé** (fichiers sur un serveur local plutôt que sur Drive). Ce travail n'est **pas fusionné dans main** et ne représente pas l'architecture actuelle du projet. À considérer seulement si cette direction est explicitement remise à l'ordre du jour.

8 Documentation

Fichier	Contenu
`README.md`	Point d'entrée : prérequis, démarrage rapide, scripts npm
`doc/TECHNICAL.md`	Référence technique vivante : architecture, API, modèle de données, conventions. À lire en premier.
`doc/DEPLOYMENT.md`	Prérequis IT pour la mise en production
`doc/APPS_SCRIPT_SYNC_SETUP.md`	Déploiement et sécurité de la synchronisation Google Apps Script
`doc/VISUAL_IDENTITY.md`	Charte graphique et conventions d'interface
`doc/FEATURE_IDEAS.md`	Idées et améliorations en attente
`SECURITY.md`	Détails de sécurité et checklist de production

9 Pour bien démarrer

1. Lire `doc/TECHNICAL.md` en entier avant de toucher au code. C'est la référence, tenue à jour à chaque changement d'architecture.
2. Faire tourner le projet en local (§5) et se connecter avec le compte admin seedé pour explorer l'interface avant de lire le code.
3. Consulter `doc/FEATURE_IDEAS.md` pour voir ce qui est déjà identifié comme prochaine étape.
4. Travailler à partir de `main` (§7) sauf instruction contraire.